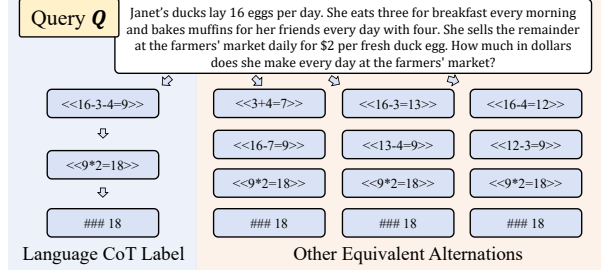


Beyond Imitation: Reinforcement Learning for Active Latent Planning

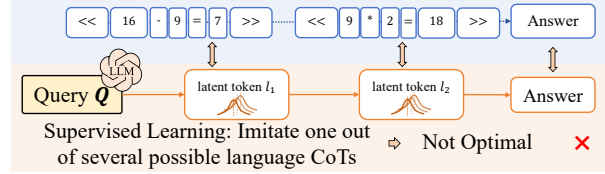
Zhi Zheng¹ Wee Sun Lee¹

Abstract

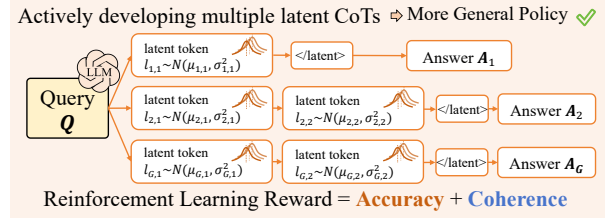
Aiming at efficient and dense chain-of-thought (CoT) reasoning, latent reasoning methods fine-tune Large Language Models (LLMs) to substitute discrete language tokens with continuous latent tokens. These methods consume fewer tokens compared to the conventional language CoT reasoning and have the potential to plan in a dense latent space. However, current latent tokens are generally supervised based on imitating language labels. Considering that there can be multiple equivalent but diverse CoT labels for a question, passively imitating an arbitrary one may lead to inferior latent token representations and latent reasoning policies, undermining the potential planning ability and resulting in clear gaps between training and testing. In this work, we emphasize the importance of active planning over the representation space of latent tokens in achieving the optimal latent reasoning policy. So, we propose the Active Latent Planning method (ATP-Latent), which models the supervision process of latent tokens as a conditional variational auto-encoder (VAE) to obtain a smoother latent space. Moreover, to facilitate the most reasonable latent reasoning policy, ATP-Latent conducts reinforcement learning (RL) with an auxiliary coherence reward, which is calculated based on the consistency between VAE-decoded contents of latent tokens, enabling a guided RL process. In experiments on LLaMA-1B, ATP-Latent demonstrates +4.1% accuracy and -3.3% tokens on four benchmarks compared to advanced baselines. Codes are available on <https://github.com/zz1358m/ATP-Latent-master>.



(a) Equivalent Language CoTs for a question



(b) Existing Imitation-based Latent Reasoning methods



(c) Active Planning for better latent reasoning policies (Ours)

Figure 1. Equivalent Language CoTs may lead to different latent reasoning policies. Existing latent reasoning methods (b) imitate one of them, leading to suboptimal policies. The proposed ATP-Latent method (c) actively optimizes the latent reasoning policies in a well-defined space, employing both the verifiable accuracy of answers and the coherence of decoded latent CoTs as rewards.

1. Introduction

Large Language models (LLMs) have shown remarkable capabilities in mathematical reasoning (Li et al., 2025b), especially equipped with the chain-of-thought (CoT) prompting techniques (Wei et al., 2022). In pursuit of superior reasoning ability, LLMs can be further refined with supervised fine-tuning (SFT) (Hu et al., 2022; Xia et al., 2025) or reinforcement learning (RL) techniques (Shao et al., 2024). However, although significant improvements in reasoning ability are achieved, these techniques usually lead to significantly longer CoT reasoning paths (Liu et al., 2025b; Feng

¹School of Computing, National University of Singapore, Singapore. Correspondence to: Wee Sun Lee <leews@comp.nus.edu.sg>.

et al., 2025), which will hinder the application of LLMs to scenarios that require real-time responses.

To reduce the latency of LLM-based reasoning, recent work has explored **implicit latent reasoning**, which aims to enable LLMs to represent and reason in a continuous latent space (Hao et al., 2024; Tan et al., 2025; Shen et al., 2025; Wei et al., 2025; Deng et al., 2025). As shown in Figure 1(b), latent reasoning methods feed high-dimensional embeddings (named latent tokens)—rather than sampled discrete language tokens—into the model at the next LLM step. These methods can avoid redundant natural-language tokens (e.g., those used mainly for semantic cohesion or interpretability-oriented summarization), thus increasing the information density per token and improving the reasoning performance under a fixed token budget.

In implementing the language-to-latent token shift, existing methods generally adopt an imitation-based strategy, mapping the thinking policy of each part of the language CoTs to a latent token through token-by-token curriculum learning (Hao et al., 2024), self-distillation (Shen et al., 2025), or compression (Tan et al., 2025). These methods can achieve clear reductions in token cost. However, as shown in Figure 1 (a), there are usually several equivalent correct language CoT for each question, so learning the latent reasoning policy by imitating an arbitrary one of them will lead to overfitted latent token representations and biased latent policies, resulting in clear performance gaps between training and testing.

To achieve more generalizable latent reasoning, this paper highlights the importance of active planning over latent tokens in pursuing the optimal latent reasoning policy. To this end, as shown in Figure 1(c), we propose the two-stage **Active Latent Planning** (ATP-Latent) method. In the SFT-stage, ATP-Latent extends the original imitation-based methods to train a variational auto-encoder (Kingma & Welling, 2013) (VAE) for interpretability and a smooth latent token representation space. Unlike prior latent reasoning methods that only passively imitate a single labeled CoT trajectory, ATP-Latent actively optimizes latent reasoning policies in the RL stage, employing both the final answer’s correctness (i.e., Accuracy in Figure 1(c)) and the consistency of VAE-decoded contents (i.e., Coherence in Figure 1(c)) as rewards, providing soft constraints for the RL state space. We conduct experiments over LLaMA-1B LLMs, demonstrating that ATP-Latent achieves superior reasoning efficiency and accuracy with fewer latent tokens compared with prior SFT and imitation-only latent reasoning methods. Our main contributions are summarized as follows:

- ATP-Latent highlights the significance of active reasoning path planning in finding a good latent reasoning policy, introducing VAE and stop-head for a smoother latent token representation space.

- In the RL process, the proposed ATP-Latent method utilizes the supervised VAE decoder to decode latent tokens and employs the consistency between the VAE-decoded contents as auxiliary rewards, yielding unsupervised but beneficial RL training signals for latent planning, providing soft constraints for the RL state space.
- ATP-Latent with LLaMA-1B demonstrates 4.1% more accuracy and 3.3% fewer tokens on numerical reasoning benchmarks compared to advanced baselines.

2. Preliminaries

2.1. Language LLM Reasoning

The language reasoning process addresses a given question $Q = (q_1, \dots, q_{|Q|})$ by first generating a series of CoT language reasoning tokens $R = (r_1, \dots, r_{|R|})$, followed by answer tokens $A = (a_1, \dots, a_{|A|})$. All tokens involved exist in the discrete language domain, meaning $Q, R, A \subset \mathcal{T}^*$, where $\mathcal{T}$ denotes the full set of language tokens. Both reasoning and answer tokens are produced according to the next-token prediction policy π_θ of large language models (LLMs), as modeled by:

$$p(R, A|Q) = \prod_{t=1}^{|R|} \pi_\theta(r_t | [Q, r_{1:t-1}]) \prod_{t=1}^{|A|} \pi_\theta(a_t | [Q, R, a_{1:t-1}]), \quad (1)$$

where $r_{1:t-1} = (r_1, \dots, r_{t-1})$ and $a_{1:t-1} = (a_1, \dots, a_{t-1})$; $[\cdot, \cdot]$ as well as $[\cdot, \cdot, \cdot]$ denote concatenation.

SFT for Language LLM Reasoning In SFT approaches for language-based LLM reasoning, carefully annotated CoT sequences $[R^*, A^*]$ are collected for each question Q . The LLM is then fine-tuned to maximize accuracy over all $|R^*| + |A^*|$ token positions. The effectiveness of SFT is highly dependent on the quality of the CoT labels, which are challenging to obtain for complex reasoning datasets.

RL Fine-tuning for Language LLM Reasoning RL methods—such as Group Relative Policy Optimization (GRPO) (Liu et al., 2024), Dr. GRPO (Liu et al., 2025c), DAPO (Yu et al., 2025a), and Lite PPO (Liu et al., 2025d)—sample multiple candidate CoTs $[R, A]$ per question and assess each with a reward reflecting the answer quality A . For example, in standard GRPO (Shao et al., 2024), G candidate CoTs are generated for each Q , and the objective is updated according to the relative advantage within these G samples. These RL-based approaches often surpass SFT in mathematical reasoning tasks. However, they have a tendency to improve performance by progressively generating longer CoT trajectories over time (Liu

$$\mathcal{J}_{\text{GRPO}}(\theta) = \frac{1}{G} \mathbb{E}_{\mathbf{Q} \sim \mathcal{D}, \{\hat{\mathbf{L}}\}_{g=1}^G, \{\mathbf{A}\}_{g=1}^G \sim p(\cdot, \cdot | \mathbf{Q})} \left[\sum_{g=1}^G \frac{1}{|\hat{\mathbf{L}}_g| + |\mathbf{A}_g|} \sum_{t=1}^{|\hat{\mathbf{L}}_g| + |\mathbf{A}_g|} \left(\min \left(p_{g,t} \hat{A}_g, \text{clip}(p_{g,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_g \right) \right) \right]$$

$$\hat{A}_g = \frac{f(\mathbf{A}_g) - \text{mean}(f(\mathbf{A}))_{g=1}^G}{\text{std}(f(\mathbf{A}))_{g=1}^G}, \quad p_{g,t} = \begin{cases} \frac{\pi_{\theta}(a_{g,t} | [\mathbf{Q}, \hat{\mathbf{L}}_g, (a_{g,1}, \dots, a_{g,t-1})])}{\pi_{\theta_{\text{old}}}(a_{g,t} | [\mathbf{Q}, \hat{\mathbf{L}}_g, (a_{g,1}, \dots, a_{g,t-1})])} & \text{if } t > |\hat{\mathbf{L}}_g| \\ \frac{\pi_{\theta}(\hat{l}_{g,t} | [\mathbf{Q}, (\hat{l}_{g,1}, \dots, \hat{l}_{g,t-1})])}{\pi_{\theta_{\text{old}}}(\hat{l}_{g,t} | [\mathbf{Q}, (\hat{l}_{g,1}, \dots, \hat{l}_{g,t-1})])} & \text{if } t \leq |\hat{\mathbf{L}}_g|. \end{cases} \quad (2)$$

et al., 2025c;d), a phenomenon termed "overthinking" (Sui et al., 2025), which can significantly reduce the inference efficiency of reasoning LLMs.

2.2. Latent CoT reasoning

To achieve efficient reasoning, recent works have attempted to stimulate concise language reasoning (Feng et al., 2025; Liu et al., 2025b; Xia et al., 2025). As another more effective but difficult way to solve overthinking, latent CoT reasoning methods (Chen et al., 2025b) aim at shifting the original reasoning process in the pre-trained language domain to $|\mathbf{L}|$ continuous latent tokens $\mathbf{L} = (l_1, \dots, l_{|\mathbf{L}|})$. Every latent token $l_i \in \mathcal{R}^d$, $i \in \{1, \dots, |\mathbf{L}|\}$ is a d -dimensional real vector. The latent reasoning LLMs still generate the answer part $\mathbf{A}$ in natural language after $\mathbf{L}$. In generating latent tokens, most latent reasoning methods follow the deterministic next(-latent)-token-prediction paradigm as follows:

$$\forall t \leq |\mathbf{L}|, \quad l_t = \pi_{\theta}([\mathbf{Q}, l_{1:t-1}]), \quad (3)$$

where $l_{1:t-1} = (l_1, \dots, l_{t-1})$. As shown in Figure 1(b), most existing approaches to latent CoT reasoning fine-tune pre-trained LLMs to imitate language labels, effectively mapping latent tokens to mimic the role of one or more language tokens (Wang et al., 2025a; Cheng & Van Durme, 2024; Shen et al., 2025). Hao et al. (2024) first enables language-to-latent domain transformation by progressively substituting original tokens in $\mathbf{R}$ with latent tokens, following a curriculum-based strategy. In the k -th curriculum stage, it substitutes language tokens in the first k steps with $c * k$ latent tokens $l_1, \dots, l_{c*k}$ (c latent tokens for each step) by minimizing the loss of cross-entropy (CE) to predict the rest of the part. This process fits well with the purpose of predicting the next-latent-token. Besides, Shen et al. (2025) fine-tunes LLM via self-distillation to encourage several latent tokens imitating the hidden state of the final token, Tan et al. (2025) and Deng et al. (2025) directly compress the embeddings of several consensus language tokens, Wei et al. (2025) introduces supervision on latent tokens, training another LLM to explain latent tokens back to language steps it imitates.

These methods offer a clear reduction in token cost. As another possible advantage on effectiveness (Hao et al., 2024), reasoning in the latent token space can effectively eliminate

some redundant tokens that are purely for syntax or explanation, thus implementing dense CoT reasoning. However, as shown in Figure 1(a), there may be several correct CoTs for one question, so imitating the implicit reasoning policy from an arbitrary label will lead to inferior latent reasoning policies and token representations, demonstrating a clear gap between training and testing. So, to find the most general latent reasoning policy, this paper proposes to highlight the planning ability of latent reasoning LLMs after active latent reasoning path exploration via RL.

3. Studies on Exploring Latent CoTs

As the main obstacle to latent reasoning exploration, current latent reasoning methods usually predict a deterministic latent token l_t at each step. So, in reinforcing latent policies with exploration, recent works Butt et al. (2025); Tan et al. (2025) introduce Gaussian Noise as $\hat{l}_t = l_t + \mathcal{N}(0, \sigma^2 I_d)$ on latent tokens and do GRPO over the latent tokens with Gaussian reparameterization. The GRPO objective of these methods is shown in Eq. (2), where $\hat{A}_g$ represents the advantage function for the g -th latent CoT $\hat{\mathbf{L}}_g = (\hat{l}_{g,1}, \dots, \hat{l}_{g,|\hat{\mathbf{L}}_g|})$, the reward function $f(\mathbf{A}_g) = 1$ if and only if the answer $\mathbf{A}_g$ is correct, ϵ is the clipping hyperparameter, $\pi_{\theta_{\text{old}}}$ is the sampling policy before updates, and we follow Yu et al. (2025a) and Tan et al. (2025) in removing the KL term to facilitate new latent policies besides the one from SFT. The reparameterization probability is calculated as follows:

$$\pi_{\theta}(\hat{l}_t | [\mathbf{Q}, l_{1:t-1}]) \propto \exp \left(-\frac{1}{2\sigma^2} \|\hat{l}_t - l_t\|_2^2 \right), \quad (4)$$

where $l_t = \pi_{\theta}([\mathbf{Q}, (\hat{l}_1, \dots, \hat{l}_{t-1})])$.

However, these methods do not demonstrate clear improvements in accuracy in primary settings (Tan et al., 2025). This might be due to their failure to maintain a smooth state space (i.e., the latent token representation space) for RL. Moreover, these methods rely on random exploration over the latent space without any confines. Both drawbacks make it hard for RL to explore a good latent reasoning policy.

In demonstrating this, as shown in Figure 3, we implement the standard GRPO in Eq. 2 over Coconut (Hao et al., 2024) and its upgrade SIM-CoT (Wei et al., 2025), which introduces a decoder to recover the imitation. Surprisingly, with

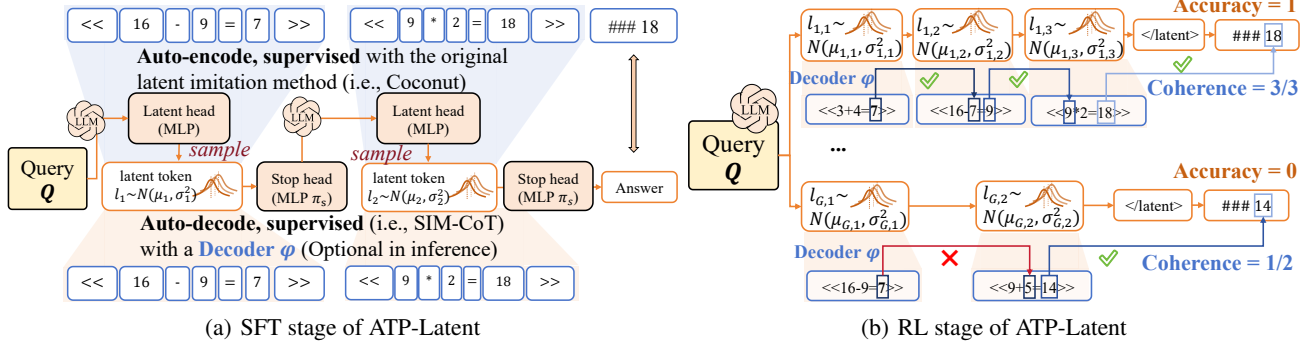


Figure 2. Equivalent Language CoTs represent different reasoning policies. Existing latent reasoning methods (b) imitate one out of them, leading to suboptimal latent policies. The proposed ATP-Latent method (c) actively optimizes latent policies, employing both the accuracy of answers and the coherence of latent CoTs as reward.

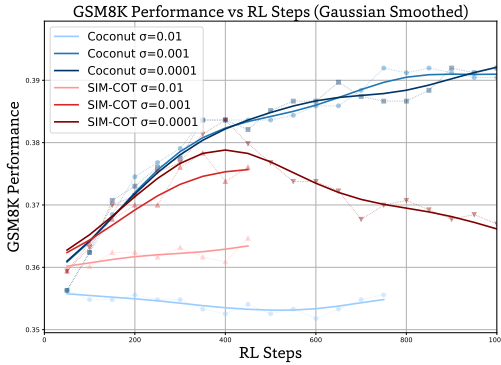


Figure 3. GRPO validation curve over Coconut and SIM-CoT and noises in different scales.

appropriate σ settings, Coconut can achieve a significant improvement through simple reparameterization and RL. However, its upgraded version, SIM-CoT, despite offering better latent space supervision and much better benchmark performances, fails to show stable benefits with RL. However, as shown in Figure 4, running SIM-CoT on a pre-trained Coconut before the RL process activates its exploration capabilities. Therefore, we attribute this difference to the SIM-CoT sharpening of the latent token representation space, thus limiting the exploration capabilities.

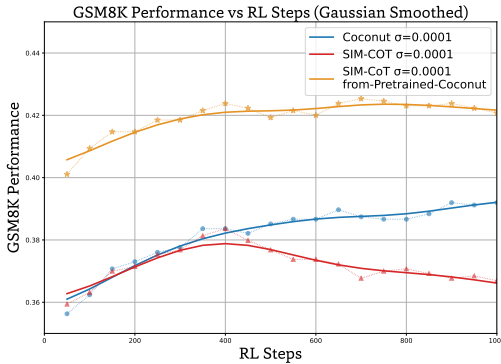


Figure 4. GRPO validation curve of SIM-CoT finetuned after Coconut training.

So, to provide a **smooth** latent token representation space conducive to RL exploration, and to leverage the explainability introduced in SIM-CoT (Wei et al., 2025) for a well-defined RL action space, we propose the ATP-Latent method. Specifically, ATP-Latent: **1)** Models the latent imitation process by training a VAE (Higgins et al., 2017) over language CoT labels, promoting uniformity across the representation space of latent tokens; **2)** Employs an automatic stopping strategy during latent token generation, ensuring that each latent token encodes a comparable amount of information and thereby unifies the latent space; **3)** Utilizes the VAE decoder to decode the latent tokens in the RL stage and introduces an unsupervised coherence reward to judge the consistency of generated contents, thus providing soft constraints on the RL state space.

4. Method: Active Latent Planning

In this section, we propose the Active Latent Planning (ATP-Latent) method (shown in Figure 2). It contains an SFT stage and a follow-up RL stage. In the SFT stage, ATP-Latent models the imitation process of the language label by **VAE-like training**, taking the LLM reasoning pipeline as the encoder and an additional LLM as the decoder for revealing each latent token to languages. We also enable the **stop mechanism** for smooth latent representations. In the following RL stage, ATP-Latent not only takes the accuracy of answers as a reward, but also measures the **coherence of the latent tokens** by the consistency between VAE-decoded contents, providing soft constraints on the RL state space.

4.1. SFT Stage

The SFT stage of ATP-Latent aims to maintain a smooth latent token representation space for the subsequent RL stage. To achieve this goal, we extend the latent imitation with the explainer idea in SIM-CoT (Wei et al., 2025) to imitate language tokens as training VAEs.

Encoder ATP-Latent adopts the original imitation-based latent reasoning model (with parameter θ) as the VAE encoder, and following Wei et al. (2025), we use another LLM (with parameter ϕ) as the VAE decoder. When using the Coconut (Hao et al., 2024) method as the ATP-Latent imitation process, in the k -th curriculum stage, it removes the first k steps (noted r_1 to $r_{n(k)}$) in the CoT language label $\mathbf{R}$, generates $c * k$ latent tokens $\mathbf{l}_1, \dots, \mathbf{l}_{c*k}$ in latent token representation space by, and minimizes the CE loss of the rest part ($r_{n(k)+1}$ to $r_{|\mathbf{R}|}$) as follows:

$$\begin{aligned} \mathcal{L}_{\text{Enc}} = & - \sum_{t=n(k)+1}^{|\mathbf{R}|} \log \pi_{\theta}(r_t | [\mathbf{Q}, \mathbf{l}_{1:c*k}, \mathbf{r}_{n(k)+1:t-1}]) \\ & - \sum_{t=1}^{|\mathbf{A}|} \log \pi_{\theta}(a_t | [\mathbf{Q}, \mathbf{l}_{1:c*k}, \mathbf{r}_{n(k)+1:|\mathbf{R}|}, \mathbf{a}_{1:t-1}]). \end{aligned} \quad (5)$$

The original Coconut and SIM-CoT generate deterministic latent tokens $\mathbf{L}$, which may cause the latent token representation to overfit the label, undermining the expression of situations in Figure 1 where there may be multiple equivalent latent policies, thus hindering the subsequent RL finetuning. So, in implementing the Gaussian distributions for VAE, we add a Multi-Layer Perceptron (MLP) as the latent head over the output hidden state, predicting the mean $\mu_t \in \mathcal{R}^d$ and standard deviation $\sigma_t \in \mathcal{R}^d$ by the latent reasoning LLM with parameter θ as follows:

$$\forall t \leq |\mathbf{L}|, \quad \mu_t, \sigma_t = \theta([\mathbf{Q}, (\mathbf{l}_1, \dots, \mathbf{l}_{t-1})]). \quad (6)$$

Then, latent tokens are sampled accordingly as follows:

$$\forall t \leq |\mathbf{L}|, \quad \mathbf{l}_t \sim \mathcal{N}(\mu_t, \text{diag}(\sigma_t^2)). \quad (7)$$

Stop Head of Encoder Most latent reasoning methods choose to generate fixed-length latent tokens during the SFT stage, which may lead to inconsistencies in the information density between latent tokens. To achieve a more uniform and smooth latent token representation space for the following planning stage, ATP-Latent designs to implement the stop policy in the CoT label $\mathbf{R}$ with K steps. We add one MLP π_{θ_s} as the stop head on each sampled latent token $\mathbf{l}_t$ for $\pi_{\theta_s}(\text{stop}|\mathbf{l}_t)$ and $\pi_{\theta_s}(\text{cont.}|\mathbf{l}_t)$, training the head only to choose stop after the K step with a Binary CE (BCE) loss $\mathcal{L}_{\text{Stop}}$ as follows:

$$\begin{aligned} \mathcal{L}_{\text{Stop}} = & - \sum_{t=1}^{\min(K, c*k)-1} \log \pi_{\theta_s}(\text{cont.}|\mathbf{l}_t) \\ & - \sum_{t=\min(K, c*k)}^{c*k} \log \pi_{\theta_s}(\text{stop}|\mathbf{l}_t). \end{aligned} \quad (8)$$

Decoder For each of the k stages. The decoder takes the series of c generated latent tokens as input, aiming

at recovering the original language label $\mathbf{r}_{n(t-1)+1:n(t)} = (r_{n(t-1)+1}, \dots, r_{n(t)})$. It optimizes the CE Loss as follows:

$$\begin{aligned} \mathcal{L}_{\text{Dec}} = & - \sum_{t=1}^k \sum_{m=n(t-1)+1}^{n(t)} \log \pi_{\phi}(r_m | [\mathbf{l}_{(t-1)*c+1:t*c}, \mathbf{r}_{n(t-1)+1:m-1}]). \end{aligned} \quad (9)$$

When $k > K$, there is no content to decode, so the decoder needs to output an empty string. As the standard setting in the VAE training process, we also optimize the KL loss based on the ELBO (Kingma & Welling, 2013) as follows:

$$\begin{aligned} \mathcal{L}_{\text{KL}} = & D_{\text{KL}}(\mathcal{N}(\mu_t, \sigma_t^2 Id) \| \mathcal{N}(0, I)) \\ = & \frac{1}{2} \sum_{t=1}^{c*k} \sum_{j=1}^d (\mu_{t,j}^2 + \sigma_{t,j}^2 - 1 - \log \sigma_{t,j}^2). \end{aligned} \quad (10)$$

Finally, the loss function in the SFT stage is designed with the β -VAE technique (Higgins et al., 2017) as follows:

$$\mathcal{L}_{\text{SFT}} = \mathcal{L}_{\text{Enc}} + \mathcal{L}_{\text{Dec}} + \mathcal{L}_{\text{Stop}} + \beta \mathcal{L}_{\text{KL}}. \quad (11)$$

For $\mathcal{L}_{\text{Dec}}$, $\mathcal{L}_{\text{Stop}}$ and $\mathcal{L}_{\text{KL}}$, we additionally normalize them by dividing the number of stages k .

The latent space often represents infinitely different latent tokens (representing infinite categories of reasoning steps). Moreover, as shown in Figure 3, each latent token tends to have only a small range of mutable values (far different from the prior $\mathcal{N}(0, I)$). So, taking into account these properties, we set a very small $\beta = \frac{1e-3}{d}$ to prioritize the latent reasoning process over decoding.

4.2. RL Stage

The RL stage of ATP-Latent actively explores the representation space of each latent token trained by VAE for the best policy across another training dataset. We design a coherence indicator to provide well-defined guidance for the state space. The RL process is built based on the Eq. 2, where we introduce the stop policy π_{θ_s} to generate $|\mathbf{L}_g|$ latent tokens for each sample. We calculate the forward policy p_t probability with $\pi_{\theta_s}(\text{cont.}|\mathbf{l}_t)$ within the latent CoT and with $\pi_{\theta_s}(\text{stop}|\mathbf{l}_t)$ at the final latent token as follows:

$$p_{g,t} = \begin{cases} \frac{\pi_{\theta}(a_{g,t} | [\mathbf{Q}_g, \mathbf{L}_g, \mathbf{a}_{g,1:t-1}])}{\pi_{\theta_{\text{old}}}(a_t | [\mathbf{Q}_g, \mathbf{L}_g, \mathbf{a}_{g,1:t-1}])} & t > |\mathbf{L}_g| \\ \frac{\pi_{\theta_s}(\text{stop}|\mathbf{l}_{g,t}) \pi_{\theta}(\mathbf{l}_{g,t} | [\mathbf{Q}_g, \mathbf{l}_{g,1:t-1}])}{\pi_{\theta_{\text{old}}}(\text{stop}|\mathbf{l}_{g,t}) \pi_{\theta_{\text{old}}}(\mathbf{l}_{g,t} | [\mathbf{Q}_g, \mathbf{l}_{g,1:t-1}])} & t = |\mathbf{L}_g| \\ \frac{\pi_{\theta_s}(\text{cont.}|\mathbf{l}_{g,t}) \pi_{\theta}(\mathbf{l}_{g,t} | [\mathbf{Q}_g, \mathbf{l}_{g,1:t-1}])}{\pi_{\theta_{\text{old}}}(\text{cont.}|\mathbf{l}_{g,t}) \pi_{\theta_{\text{old}}}(\mathbf{l}_{g,t} | [\mathbf{Q}_g, \mathbf{l}_{g,1:t-1}])} & t < |\mathbf{L}_g|. \end{cases} \quad (12)$$

where $\pi_{\theta_{s,\text{old}}}$ is the stop policy in sampling, π_{θ_s} represents the current stop policy. $\pi_{\theta_{\text{old}}}$ is the sampling latent policy, π_{θ} is the current latent policy. The probability of

Table 1. Comparison of (latent) reasoning methods and their ablations on four math reasoning benchmarks (GSM8K, GAM-hard, MultiArith, SVAMP). We report the answer accuracy (noted Acc, higher is better) and the average number of generated tokens $|\mathbf{L}| + |\mathbf{A}|$ (#Token, lower is better) for each approach. We bolded the best-performing method for each benchmark and highlighted cells indicate the second-best performance in each setting. * represents the results reported in Wei et al. (2025). The proposed ATP-Latent and its ablations demonstrate the effectiveness of latent reasoning with the proposed components, including VAE, Stop head, and the RL stage. w Only Coherence as Reward represents using only the unsupervised coherence reward in RL training. We show the change of average accuracy in colors. For methods with randomness, we run each method 5 times and report the average.

	GSM8K		GAM-hard		MultiArith		SVAMP		Avg	
	Acc	#Token	Acc	#Token	Acc	#Token	Acc	#Token	Acc	#Token
CoT-SFT	52.3	30.6	12.3	35.6	94.4	19.0	58.0	16.3	54.3	25.4
Answer-SFT	26.4	4.2	6.4	4.8	49.4	4.0	35.3	4.1	29.4	4.3
CoLaR (Tan et al., 2025)	25.7	10.0	5.7	11.5	86.8	7.2	49.9	7.1	42.0	8.9
iCoT* (Deng et al., 2023)	30.1	2.2	5.7	-	55.5	-	29.4	-	30.2	-
Coconut (Hao et al., 2024)	35.6	9.2	8.2	9.8	86.1	9.0	37.0	9.1	41.7	9.3
Coconut* (Hao et al., 2024)	36.6	13.2	8.1	-	83.5	-	36.2	-	41.1	-
SIM-CoT (Wei et al., 2025)	35.9	9.2	8.6	9.8	87.8	9.0	42.0	9.0	43.6	9.2
SIM-CoT* (Wei et al., 2025)	42.2	13.2	9.3	-	87.7	-	43.9	-	45.8	-
ATP-Latent (Ours)	42.3	9.8	9.8	10.0	94.4	7.1	44.2	6.8	47.7	8.4
-w/o VAE	40.8	9.7	9.2	10.1	92.8	7.2	46.0	6.4	47.2(-0.5)	8.4
-w/o Stop Head	41.2	9.2	8.8	9.8	93.9	9.0	43.7	9.0	46.9(-0.8)	9.3
-w/o RL (Ours-SFT)	40.0	9.5	9.3	9.7	90.0	7.1	43.7	6.5	45.8(-1.9)	8.2
-w Only Coherence as Reward	40.9	9.4	9.5	9.7	90.6	7.1	46.0	6.5	46.7(+0.9)	8.2

$\pi_\theta(\mathbf{l}_t | [\mathbf{Q}, \mathbf{l}_{1:t-1}])$ is obtained with the reparameterization trick as follows:

$$\pi_\theta(\mathbf{l}_t | [\mathbf{Q}, \mathbf{l}_{1:t-1}]) = \prod_{i=1}^d \frac{1}{\sqrt{2\pi\sigma_{t,i}^2}} \exp \left[-\frac{1}{2} \frac{(\mathbf{l}_{t,i} - \mu_{t,i})^2}{\sigma_{t,i}^2} \right]. \quad (13)$$

After training the decoder ϕ in VAE, it can well translate the latent tokens to languages $\mathbf{R}'$ as follows, even with tolerable noise:

$$\forall_{i=1}^{\lceil \frac{|\mathbf{L}|}{c} \rceil} \mathbf{R}'_i = \pi_\phi(\mathbf{l}_{(i-1)*c+1}, \dots, \mathbf{l}_{i*c}). \quad (14)$$

As shown in the 2 (b), there are different latent CoTs in the GRPO-based RL sampling, some of which (the lower one) have inconsistent explanations. For math reasoning, ATP-Latent proposes to judge the coherence $R_{\text{Coh}}(\mathbf{L})$ of latent tokens with the consistency of decoded contents, counting the ratio of equations whose **results** (i.e., right-hand side (RHS)) appear in the **left-hand side** (LHS) of the following latent CoT steps or the final answer $\mathbf{A}$ as follows:

$$R_{\text{Coh}}(\mathbf{L}) = \frac{\sum_{i=1}^{|\mathbf{L}|} \mathbb{I}[\exists \mathbf{S} \in \mathbb{S} \text{ RHS}(\pi_\phi(\mathbf{L}_i)) = \mathbf{S}]}{\sum_{i=1}^{|\mathbf{L}|} 1}, \quad (15)$$

$$\mathbb{S} = \text{LHS}(\mathbf{R}'_1 \cup \dots \cup \mathbf{R}'_{\lceil \frac{|\mathbf{L}|}{c} \rceil} \cup \mathbf{A}).$$

Generally, we believe latent CoTs with consistent and reasonable explanations can show that they really participate in reasoning, thus representing better policy. Instead, inconsistency between decoded explanations may reflect a wrong latent reasoning policy or even indicate that these

latent tokens do not participate in the final answering phase. Existing methods fail to provide reliable rewards over the latent tokens, only being able to explore the general optimal latent policies randomly. ATP-Latent proposes to use the reward $R_{\text{Coh}}(\mathbf{L})$ on latent tokens for soft guidance. So, the reward $f(\mathbf{L}, \mathbf{A})$ of ATP-Latent prioritizes correct answers with latent CoTs in higher $R_{\text{Coh}}(\mathbf{L})$ as follows:

$$f(\mathbf{L}, \mathbf{A}) = (1 + 0.1 \cdot R_{\text{Coh}}(\mathbf{L})) R_{\text{Correct}}(\mathbf{A}) + 0.5 \cdot R_{\text{Format}}(\mathbf{A}). \quad (16)$$

where R_{Correct} and R_{Format} represent the binary correctness of the answer and format (please refer to Appendix B), respectively. It can provide soft constraints on the state space (i.e., latent tokens). Besides, latent reasoning methods sometimes actually build question-to-answer shortcuts instead of reasoning step-by-step (Zhang et al., 2025b), making latent tokens useless. Introducing $R_{\text{Coh}}(\mathbf{L})$ will also be beneficial in avoiding shortcuts.

Coherence as Unsupervised Reward The coherence reward can also serve as a powerful unsupervised reward. So, ATP-Latent provides an unsupervised setting in RL as well, using only the coherence of latent CoTs and format reward rewards as follows:

$$f(\mathbf{L}, \mathbf{A}) = R_{\text{Coh}}(\mathbf{L}) + 0.5 \cdot R_{\text{Format}}(\mathbf{A}). \quad (17)$$

5. Experiment

In this section, we evaluate the proposed ATP-Latent based on the *LLaMA-3.2-1B-Instruct* LLM.

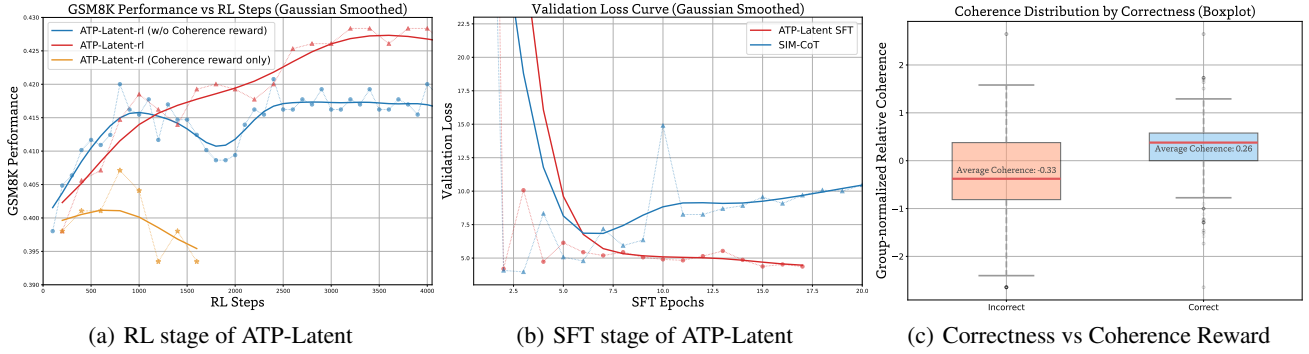


Figure 5. Ablation study and case studies on ATP-Latent. (a) exhibits the validation accuracy curve during RL on GSM8K. (b) shows the loss curve during the SFT stage on the validation set of GSM8K-Aug (500 instances). (c) presents the relationship between the correctness of questions on the training dataset (RL split) and the coherence reward before the RL training in ATP-Latent.

Training & Testing Settings Following previous work (Hao et al., 2024; Wei et al., 2025; Tan et al., 2025), we adopt the **GSM8K-Aug** (Deng et al., 2024) with 385k instances as the training dataset. As shown in Figure 1 and 2, the language CoTs in the dataset are built with equation steps. In training latent reasoning methods with RL stages, we divide the training data into two parts, 80% for SFT and 20% for RL. Detailed parameters are shown in Appendix C.

We follow (Wei et al., 2025) in adopting the GSM8K test set for in-domain, GSM-Hard (Gao et al., 2023), SVAMP (Patel et al., 2021), and Multi-Arith (Roy & Roth, 2015) for out-of-domain evaluation.

Baselines We adopt **latent reasoning methods** Coconut (Hao et al., 2024), SIM-CoT (Wei et al., 2025) (building on Coconut), iCoT (Deng et al., 2024), and CoLaR (Tan et al., 2025) as baselines. Besides, we introduce CoT-SFT for SFT with **language CoT steps** and Answer-SFT for supervising only the answer. All experiments are implemented on a node of 8× NVIDIA H200 GPUs (141 GB VRAM each). In obtaining baselines in a similar number of tokens, we set the compression factor in CoLaR to 5, $c = 2$ for three curriculum stages in Coconut, and SIM-CoT. Please refer to Appendix E for the detailed implementations of baselines.

5.1. Results

Table 1 shows the performance of our proposed ATP-Latent and representative latent-reasoning baselines on four math reasoning benchmarks. We report answer accuracy (Acc; $\times 100$ for clarity, higher is better) and the average number of generated tokens (#Token; lower is better) to jointly measure correctness and generation efficiency. As shown, ATP-Latent achieves the strongest overall trade-off, reaching 47.7% average accuracy with only 8.4 tokens on average, while being consistently competitive across all datasets; notably, it attains the outstanding 94.4% on MultiArith. Compared to SIM-CoT and Coconut baselines, where we report both the re-implementation results and results reported in

Wei et al. (2025) (noted with “*”), ATP-Latent can demonstrate a +4.1% accuracy and -3.3% tokens compared to the reimplemented SIM-CoT result, demonstrating the benefits gained from doing active planning within a well-defined and smooth latent token representation space. In Appendix D, we conduct more comparisons to baselines boosted with RL, where ATP-Latent can still demonstrate superiority.

5.2. Main Ablations

In Table 1, we also present ablation studies on the VAE (w/o VAE for removing the σ_t prediction and KL loss, running the SIM-CoT-like method with stop-head over Coconut), the stop head (w/o stop head for dropping the stop head, keeping the max stages to 3), and the RL stage (w/o RL for the ATP-Latent model after the SFT stage). The ablation results validate the effectiveness of each proposed component: removing the VAE or the Stop Head leads to consistent accuracy drops (average -0.5% and -0.8%, respectively), and removing the RL stage yields the largest degradation (average -1.9%), highlighting the importance of RL for improving latent reasoning. Furthermore, w Only Coherence as Reward RL represents the unsupervised RL method shown in Section 4.2, where using only the unsupervised coherence reward still improves the SFT-only model (average +0.9%), indicating that coherence provides a useful unsupervised training signal.

6. Discussion

In this section, we conduct more ablation studies on components, as well as case studies over the interpretability and diversity of latent planning.

6.1. More Ablation Studies

Ablations on the Reward Function In Table 1, we provide ablations over some components of ATP-Latent (i.e., VAE, Stop Head, and RL stage). In this subsection, we

discuss the importance of introducing the coherence reward R_{Coh} in the final reward $f(L, A)$. As shown in Figure 5 (a), removing the coherence reward R_{Coh} (the variant ATP-Latent-rl (without the coherence reward)) can clearly demonstrate inferior in-domain performance compared to the original ATP-Latent. It shows the significance of the coherence reward in RL training; we believe it can provide a good reweighting over positive samples, helping to find latent reasoning policies that conform to optimal properties.

Ablations on the VAE Training As shown in Figure 5(b), by modeling the imitation process as training a VAE with a stop-head, ATP-Latent can demonstrate smoother convergence on the validation loss function in SFT compared to the SIM-CoT baseline (using the Coconut backbone).

6.2. Case Study: Interpretability for Planning

In this section, we discuss the validity of using coherence as an unsupervised reward and examine the changes in the VAE-decoded content to illustrate cases of the latent reasoning policy after the RL stage of ATP-Latent.

The Validity of Coherence in Reward As shown in Figure 5 (a), using coherence as the only unsupervised reward can improve GSM8K performance in the first 800 steps. We believe that this benefits from the fact that the coherence can be a good unsupervised reward model for accuracy. As shown in Figure 5 (c), we calculate the coherence on latent tokens generated by ATP-Latent after the SFT stage. The average group-normalized coherence value is 0.26 for correct examples and -0.33 for incorrect ones, demonstrating clear differences.

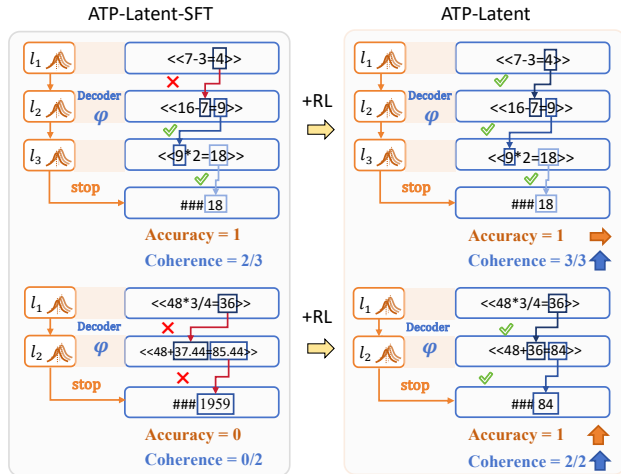


Figure 6. Examples of interpretability of latent tokens and improvements after RL. The accuracy and coherence can be improved.

Examples for the Latent Reasoning Policy Similar to Wei et al. (2025), ATP-Latent can provide optional interpretability of latent tokens, using the VAE decoder ϕ . As

shown in Figure 8, the RL process can bring more consistent steps, as well as more accurate results. The improvement in consistency can be attributed to the introduction of R_{Coh} . In the second example, LLMs will prioritize correct and more coherent latent reasoning policies, proceeding with the planning process step by step, improving the probability of finding general and superior latent reasoning policies.

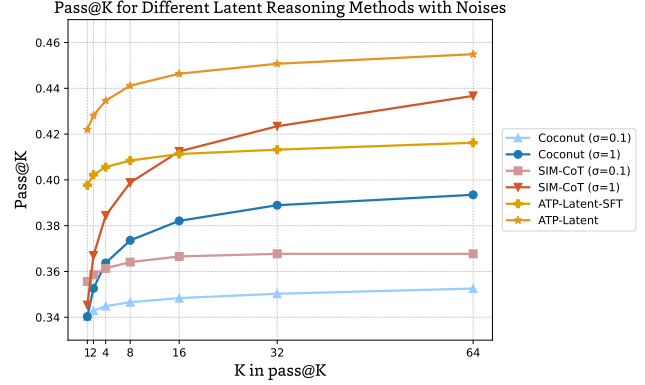


Figure 7. Pass@K curve for ATP-Latent and baselines. We run methods 64 times with Gaussian noise and different variances for Pass@K. The variances in ATP-Latent are predicted by itself.

6.3. Incentivize Planning Capacity Beyond Supervision

To investigate whether the RL stage of ATP-Latent can incentivize the latent planning ability, we sample 64 runs with different L for Pass@K (Yue et al., 2025a). Pass@K judges whether the correct answer occurs in every K runs. As shown in Figure 7, after the RL training process, compared to baselines and the ATP-Latent-SFT, ATP-Latent can show significantly higher Pass@K from $K=1$ to 64. It can reflect that ATP-Latent can develop latent planning capacities in the RL stage (examples are given in Appendix D.3).

7. Conclusion

In this work, we highlight the significance of active planning over the latent token representation space and introduce ATP-Latent for actively planning in a well-defined and smooth latent space. Unlike prior latent reasoning methods that rely heavily on imitating single annotated CoT trajectories, ATP-Latent leverages a VAE framework and introduces a stop-generation mechanism to ensure a smoother and more expressive latent space. Moreover, by incorporating both the accuracy and coherence of VAE-decoded latent CoTs as rewards, our method provides soft constraints on the RL state space for more generalizable latent reasoning policies. Evaluations on four math reasoning benchmarks demonstrate that ATP-Latent achieves higher accuracy with significantly fewer generated tokens compared to existing baselines. ATP-Latent provides a promising direction for latent reasoning, and we will implement it on more tasks (e.g., natural language and complex LaTeX) in the future.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Butt, N., Kwiatkowski, A., Labiad, I., Kempe, J., and Ollivier, Y. Soft tokens, hard truths. *arXiv preprint arXiv:2509.19170*, 2025.
- Chen, X., Zhao, A., Xia, H., Lu, X., Wang, H., Chen, Y., Zhang, W., Wang, J., Li, W., and Shen, X. Reasoning beyond language: A comprehensive survey on latent chain-of-thought reasoning, 2025a. URL <https://arxiv.org/abs/2505.16782>.
- Chen, X., Zhao, A., Xia, H., Lu, X., Wang, H., Chen, Y., Zhang, W., Wang, J., Li, W., and Shen, X. Reasoning beyond language: A comprehensive survey on latent chain-of-thought reasoning. *arXiv preprint arXiv:2505.16782*, 2025b.
- Cheng, J. and Van Durme, B. Compressed chain of thought: Efficient reasoning through dense representations. *arXiv preprint arXiv:2412.13171*, 2024.
- Deng, J., Pang, L., Wei, Z., Xu, S., Duan, Z., Xu, K., Song, Y., Shen, H., and Cheng, X. Latent reasoning in llms as a vocabulary-space superposition. *arXiv preprint arXiv:2510.15522*, 2025.
- Deng, Y., Prasad, K., Fernandez, R., Smolensky, P., Chaudhary, V., and Shieber, S. Implicit chain of thought reasoning via knowledge distillation, 2023.
- Deng, Y., Choi, Y., and Shieber, S. From explicit cot to implicit cot: Learning to internalize cot step by step. *arXiv preprint arXiv:2405.14838*, 2024.
- Dong, S., Wang, S., Liu, X., Li, C., Hou, H., and Wei, Z. Interleaved latent visual reasoning with selective perceptual modeling. *arXiv preprint arXiv:2512.05665*, 2025.
- Feng, S., Fang, G., Ma, X., and Wang, X. Efficient reasoning models: A survey. *arXiv preprint arXiv:2504.10903*, 2025.
- Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., and Neubig, G. Pal: Program-aided language models. In *International Conference on Machine Learning*, pp. 10764–10799. PMLR, 2023.
- Goyal, S., Ji, Z., Rawat, A. S., Menon, A. K., Kumar, S., and Nagarajan, V. Think before you speak: Training language models with pause tokens. In *ICLR*, 2024. URL <https://openreview.net/forum?id=k5ElYw5u3Q>.
- Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., and Tian, Y. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- Hendrycks, D. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., and Lerchner, A. beta-vae: Learning basic visual concepts with a constrained variational framework. In *International conference on learning representations*, 2017.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Hu, Z., Wang, Y., He, Y., Wu, J., Zhao, Y., Ng, S.-K., Breazeal, C., Luu, A. T., Park, H. W., and Hooi, B. Rewarding the rare: Uniqueness-aware rl for creative problem solving in llms. *arXiv preprint arXiv:2601.08763*, 2026.
- Kingma, D. P. and Welling, M. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- Kong, D., Zhao, M., Xu, D., Pang, B., Wang, S., Honig, E., Si, Z., Li, C., Xie, J., Xie, S., et al. Latent thought models with variational bayes inference-time computation. *arXiv preprint arXiv:2502.01567*, 2025.
- Li, B., Sun, X., Liu, J., Wang, Z., Wu, J., Yu, X., Chen, H., Barsoum, E., Chen, M., and Liu, Z. Latent visual reasoning. *arXiv preprint arXiv:2509.24251*, 2025a.
- Li, Z.-Z., Zhang, D., Zhang, M.-L., Zhang, J., Liu, Z., Yao, Y., Xu, H., Zheng, J., Wang, P.-J., Chen, X., et al. From system 1 to system 2: A survey of reasoning large language models. *arXiv preprint arXiv:2502.17419*, 2025b.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Liu, J., Huang, Z., Sims, A., Chen, E., Teh, Y. W., and Miao, N. Marcos: Deep thinking by markov chain of continuous thoughts. *arXiv preprint arXiv:2509.25020*, 2025a.
- Liu, Y., Wu, J., He, Y., Gong, R., Xia, J., Li, L., Gao, H., Chen, H., Bi, B., Zhang, J., et al. Efficient inference for large reasoning models: A survey. *arXiv preprint arXiv:2503.23077*, 2025b.
- Liu, Z., Chen, C., Li, W., Qi, P., Pang, T., Du, C., Lee, W. S., and Lin, M. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025c.

- Liu, Z., Liu, J., He, Y., Wang, W., Liu, J., Pan, L., Hu, X., Xiong, S., Huang, J., Hu, J., et al. Part i: Tricks or traps? a deep dive into rl for llm reasoning. *arXiv preprint arXiv:2508.08221*, 2025d.
- Ma, J., Zhou, X., Song, Y., and Yan, H. Multi-modal reasoning via latent refocusing. *arXiv preprint arXiv:2511.02360*, 2025.
- Ning, A., Kuo, Y.-L., and Gomes, G. Learning when to stop: Adaptive latent reasoning via reinforcement learning. *arXiv preprint arXiv:2511.21581*, 2025.
- Özeren, E. and Aßenmacher, M. Reinforcement learning for latent-space thinking in llms. *arXiv preprint arXiv:2512.11816*, 2025.
- Patel, A., Bhattamishra, S., and Goyal, N. Are nlp models really able to solve simple math word problems? *arXiv preprint arXiv:2103.07191*, 2021.
- Pham, T.-H. and Ngo, C. Multimodal chain of continuous thought for latent-space reasoning in vision-language models. *arXiv preprint arXiv:2508.12587*, 2025.
- Qin, Y., Wei, B., Ge, J., Kallidromitis, K., Fu, S., Darrell, T., and Wang, X. Chain-of-visual-thought: Teaching vlms to see and think better with continuous visual tokens. *arXiv preprint arXiv:2511.19418*, 2025.
- Roy, S. and Roth, D. Solving general arithmetic word problems. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pp. 1743–1752, 2015.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shen, Z., Yan, H., Zhang, L., Hu, Z., Du, Y., and He, Y. Codi: Compressing chain-of-thought into continuous space via self-distillation. *arXiv preprint arXiv:2502.21074*, 2025.
- Sui, Y., Chuang, Y.-N., Wang, G., Zhang, J., Zhang, T., Yuan, J., Liu, H., Wen, A., Zhong, S., Zou, N., et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*, 2025.
- Tan, W., Li, J., Ju, J., Luo, Z., Luan, J., and Song, R. Think silently, think fast: Dynamic latent compression of llm reasoning chains. *arXiv preprint arXiv:2505.16552*, 2025.
- Wang, J., Wu, Z., Lai, F., Lian, S., and Zeng, Z. Synadapt: Learning adaptive reasoning in large language models via synthetic continuous chain-of-thought. *arXiv preprint arXiv:2508.00574*, 2025a.
- Wang, Q., Shi, Y., Wang, Y., Zhang, Y., Wan, P., Gai, K., Ying, X., and Wang, Y. Monet: Reasoning in latent visual space beyond images and language. *arXiv preprint arXiv:2511.21395*, 2025b.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Wei, X., Liu, X., Zang, Y., Dong, X., Cao, Y., Wang, J., Qiu, X., and Lin, D. Sim-cot: Supervised implicit chain-of-thought. *arXiv preprint arXiv:2509.20317*, 2025.
- Wu, C., Lu, J., Ren, Z., Hu, G., Wu, Z., Dai, D., and Wu, H. Llms are single-threaded reasoners: Demystifying the working mechanism of soft thinking. *arXiv preprint arXiv:2508.03440*, 2025.
- Xia, H., Leong, C. T., Wang, W., Li, Y., and Li, W. Token-skip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*, 2025.
- Xu, Y., Guo, X., Zeng, Z., and Miao, C. Softcot: Soft chain-of-thought for efficient reasoning with llms, 2025a.
- Xu, Y., Guo, X., Zeng, Z., and Miao, C. Softcot++: Test-time scaling with soft chain-of-thought reasoning. *arXiv preprint arXiv:2505.11484*, 2025b.
- You, R., Li, Y., Liu, M., Wang, W., Nie, L., and Li, W. Parallel test-time scaling for latent reasoning models. *arXiv preprint arXiv:2510.07745*, 2025.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025a.
- Yu, X., Xu, C., Zhang, G., Chen, Z., Zhang, Y., He, Y., Jiang, P.-T., Zhang, J., Hu, X., and Yan, S. Vismem: Latent vision memory unlocks potential of vision-language models. *arXiv preprint arXiv:2511.11007*, 2025b.
- Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Song, S., and Huang, G. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025a.
- Yue, Z., Jin, B., Zeng, H., Zhuang, H., Qin, Z., Yoon, J., Shang, L., Han, J., and Wang, D. Hybrid latent reasoning via reinforcement learning. *arXiv preprint arXiv:2505.18454*, 2025b.
- Zelikman, E., Harik, G. R., Shao, Y., Jayasiri, V., Haber, N., and Goodman, N. Quiet-star: Language models can teach themselves to think before speaking. In *First Conference*

on *Language Modeling*, 2024. URL <https://arxiv.org/abs/2403.09629>.

Zhang, J., Zhu, Y., Sun, M., Luo, Y., Qiao, S., Du, L., Zheng, D., Chen, H., and Zhang, N. Lightthinker: Thinking step-by-step compression. *arXiv preprint arXiv:2502.15589*, 2025a.

Zhang, Y., Tang, B., Ju, T., Duan, S., and Liu, G. Do latent tokens think? a causal and adversarial analysis of chain-of-continuous-thought. *arXiv preprint arXiv:2512.21711*, 2025b.

Zhang, Z., He, X., Yan, W., Shen, A., Zhao, C., Wang, S., Shen, Y., and Wang, X. E. Soft thinking: Unlocking the reasoning potential of llms in continuous concept space. *arXiv preprint arXiv:2505.15778*, 2025c.

Zheng, Y., Zhang, R., Zhang, J., Ye, Y., Luo, Z., Feng, Z., and Ma, Y. Llamafactory: Unified efficient fine-tuning of 100+ language models. *arXiv preprint arXiv:2403.13372*, 2024.

Zheng, Z. and Lee, W. S. Soft-grpo: Surpassing discrete-token llm reinforcement learning via gumbel-reparameterized soft-thinking policy optimization. *arXiv preprint arXiv:2511.06411*, 2025.

Zhu, R.-J., Peng, T., Cheng, T., Qu, X., Huang, J., Zhu, D., Wang, H., Xue, K., Zhang, X., Shan, Y., et al. A survey on latent reasoning. *arXiv preprint arXiv:2507.06203*, 2025.

Zhuang, Y., Liu, L., Singh, C., Shang, J., and Gao, J. Text generation beyond discrete token sampling. *arXiv preprint arXiv:2505.14827*, 2025.

A. Related Work

A.1. Latent Reasoning

Latent reasoning methods decouple the reasoning process from explicit natural language and perform inference in the hidden space of the model. Existing latent methods can be generally divided into two categories: auto-aggressive methods and auxiliary strategies (Chen et al., 2025a; Zhu et al., 2025). Token-wise auto-aggressive methods transform the conventional discrete-token CoT reasoning process into generating reasoning chains with dense latent embeddings (Hao et al., 2024) or specialized tokens (e.g., pause (Goyal et al., 2024; Zelikman et al., 2024)) one-by-one. These methods focus on transferring the original reasoning policy in the language domain to a latent embedding space, including curriculum learning (e.g., Coconut (Hao et al., 2024), LightThinker (Zhang et al., 2025a), self-distillation (e.g., CODI (Shen et al., 2025)), and one-shot compression (e.g., CoLaR (Tan et al., 2025), SynAdapt (Wang et al., 2025a)). Due to the existing token-wise auto-aggressive methods completely treating the language CoTs as the label, these methods can hardly surpass or even reach the performance level of discrete-token CoT reasoning. So empirically, these methods can effectively reduce the token cost compared to language CoT, but clear performance drop.

Latent reasoning methods with auxiliary strategies (e.g., SoftCoT (Xu et al., 2025a)) generate latent embeddings from an auxiliary module and inject them into the main model for the following discrete-token CoT (Cheng & Van Durme, 2024; Xu et al., 2025b; Kong et al., 2025; Liu et al., 2025a). Unlike auto-regressive latent reasoning methods, auxiliary strategies can effectively improve the performance of the original LLM, but at the cost of running efficiency.

This paper focuses on designing a token-wise auto-aggressive latent reasoning method. The proposed ATP-Latent highlights the fact that the optimal latent reasoning policy can only be obtained by imitation, and first builds the step-by-step imitation process as a VAE.

A.2. Latent Reasoning with Reinforcement Learning

As briefly illustrated in Section 3, there are several existing works or contemporary on using RL for latent reasoning (Tan et al., 2025; Özeren & Aßenmacher, 2025; Ning et al., 2025). These methods employ Gaussian noise and reparameterization techniques, addressing the issues of latent tokens often being deterministic, lacking randomness, and the gradient problem for RL. These methods often rely on searching for the optimal latent reasoning policy within a given Gaussian space using batch data. In contrast, ATP-Latent emphasizes using coherence to first orthogonalize a defined legitimate latent planning approach, avoiding shortcuts (Zhang et al., 2025b; Shen et al., 2025) that directly answer without considering intermediate latent reasoning steps.

Besides RL methods, some other methods incorporate noise for planning, You et al. (2025) propose to do test-time compute, using a fine-tuned verifier for majority voting. Although accuracy improvements are made, this method will harm the effective nature of latent reasoning.

There are also other reinforcement learning methods for latent reasoning methods with auxiliary strategies. Yue et al. (2025b) fine-tunes LLMs to pass hidden states between tokens to enrich the LLM’s discrete-token reasoning.

A.3. Soft-Thinking Reasoning

As a special category of reasoning method, soft-thinking reasoning passes the summation of token embeddings weighted by output probabilities as the LLM step. It mainly aims at conveying abstract concepts that cannot be represented by one language token and improving information density (Zhang et al., 2025c; Wu et al., 2025; Zhuang et al., 2025; Butt et al., 2025; Zheng & Lee, 2025). Therefore, although soft-thinking can demonstrate better performance than discrete-token in many cases, and the RL method Soft-GRPO based on the soft-thinking reasoning pattern can show better Pass@K performance than discrete-token GRPO, such methods cannot lead to the significant efficiency improvements seen in latent-reasoning.

A.4. Latent Reasoning in Vision Language Model

Recent work also tries to adopt the latent reasoning idea in visual tasks (Li et al., 2025a; Wang et al., 2025b; Pham & Ngo, 2025; Chen et al., 2025b; Ma et al., 2025; Qin et al., 2025; Yu et al., 2025b; Dong et al., 2025), and some of them adopt the RL idea in Tan et al. (2025). But as far as we acknowledge, none of them adopts the same coherence idea in the proposed ATP-Latent.

B. Prompt for ATP-Latent

In this section, we will introduce the format for ATP-Latent as well as the settings for the format reward R_{Format} . The prompt for latent reasoning contains only the question as follows:

Prompt for ATP-Latent.

user: Janet’s ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers’ market daily for \$2 per fresh duck egg. How much in dollars does she make every day at the farmers’ market? Let’s think step by step and output the final answer after ### <latent>

assistant: ...Latent tokens ... </latent>### 18

We set R_{Format} as 1 if the ### can be successfully output after <latent> and 0 otherwise. The decoder ϕ inputs only the c latent tokens in one step for the decoded content.

In Table 1, the prompts of CoT-SFT and Answer-SFT baselines are as follows.

Prompt for CoT-SFT in Tabel 1

user: Janet’s ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers’ market daily for \$2 per fresh duck egg. How much in dollars does she make every day at the farmers’ market? Let’s think step by step and output the final answer after ###

assistant: ...Language CoT ...### 18

Prompt for Answer-SFT in Tabel 1

user: Janet’s ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder at the farmers’ market daily for \$2 per fresh duck egg. How much in dollars does she make every day at the farmers’ market? Output the final answer after ###

assistant: ### 18

In counting the number of generated tokens of all methods, we consider all the tokens generated by LLMs (i.e., except <latent>, </latent>).

C. Detailed parameters

The detailed parameters of ATP-Latent are listed as follows. We use the *LLaMA-3.2-1B-Instruct* LLM for both the reasoning LLM θ and decoder ϕ . Both the stop head and the latent head are two-layer MLPs using the GELU (Hendrycks, 2016) activation.

Table 2. Parameters of ATP-Latent

Parameter	Value
ATP-LATENT SFT stage	
Optimizer	AdamW
Batch size	256
Learning rate	1e-4
KL loss factor β (KL normalized with d)	1e-3
Weight decay	0.01
Maximum stages	10
Number of epochs per stage	2
Number of latent tokens for each stage c	2
Number of epochs	15
GPU usage	8
Data percentage	80%
ATP-LATENT RL stage	
Optimizer	AdamW
Batch size	16
Learning rate	1e-6
Maximum answer length	16 tokens
Maximum stages	10
Number of latent tokens for each stage c	2
Sampling temperature	1.0
(top-p, top-k)	(1, -1)
Number of epochs	1
Clipping threshold ϵ	0.2
KL loss factor in GRPO (Shao et al., 2024)	0
GPU usage	1
Data percentage	20%
ATP-LATENT Testing	
Batch size	1
Maximum answer length	16 tokens
Maximum stages	10
Number of latent tokens for each stage c	2
Sampling temperature	0
(top-p, top-k)	(1, -1)

D. Supplementary Experiments

D.1. Comparison with Baselines + RL

Table 1 demonstrates the performance of ATP-Latent. In this part, we compare ATP-Latent with more latent reasoning methods and the reinforcement learning (RL) fine-tuning. Metrics include accuracy (Acc) and average reasoning length (#Token).

Results in Table 3 show that RL fine-tuning brings little improvement for CoLaR, but leads to substantial performance gains for Coconut, Coconut + SIM-CoT (fine-tuning for SIM-CoT over a pre-trained Coconut, the one shown in Figure 4), and especially the proposed ATP-Latent method. After RL tuning, ATP-Latent achieves the highest overall average accuracy (47.7%), outperforming all baselines. The RL process of ATP-Latent demonstrates more significant improvement compared to the Coconut + SIM-CoT baseline, indicating the VAE, stop-head, and coherence reward can provide better state space for the RL improvement.

Table 3. Comparison of latent reasoning methods with the following RL fine-tuning. We consider CoLaR, Coconut, Coconut + SIM-CoT (fine-tuning SIM-CoT over pretrained Coconut), and the proposed ATP-Latent.

Methods	GSM8K		GAM-hard		MultiArith		SVAMP		Avg	
	Acc	#Token	Acc	#Token	Acc	#Token	Acc	#Token	Acc	#Token
CoLaR (Tan et al., 2025)	25.7	10.0	5.7	11.5	86.8	7.2	49.9	7.1	42.0	8.9
+RL	25.5	10.0	5.9	11.4	86.8	7.2	49.0	7.1	41.8(-0.2)	8.9
Coconut (Hao et al., 2024)	35.6	9.2	8.2	9.8	86.1	9.0	37.0	9.1	41.7	9.3
+RL	40.0	9.2	9.3	9.8	90.0	9.0	42.7	9.0	45.5(+3.8)	9.3
Coconut + SIM-CoT (Wei et al., 2025)	40.0	9.2	9.7	9.8	88.9	9.0	45.3	9.0	46.0	9.3
+RL	42.1	9.2	10.2	9.8	90.6	9.0	45.7	9.0	47.1(+1.1)	9.3
ATP-Latent-SFT (Ours)	40.0	9.5	9.3	9.7	90.0	7.1	43.7	6.5	45.8	8.2
+RL (Ours)	42.3	9.8	9.8	10.0	94.4	7.1	44.2	6.8	47.7(+1.9)	8.4

D.2. Comparison on the Self-Extending Ability

As mentioned in Wei et al. (2025), the Coconut has a latent instability issue. Coconut pre-trained on a specific maximum stage (3 in their and our paper) cannot extend to larger stages, and SIM-CoT can mitigate such degradation in performance when generalizing to larger stages (i.e., larger maximum numbers of latent tokens).

ATP-Latent can totally solve such an instability issue by introducing a stop mechanism. As shown in Table 4, ATP-Latent and its SFT model will not drop performance when the number of maximum stages (i.e., the maximum number of latent tokens) improves.

Table 4. The performance on GSM8K of Coconut, SIM-CoT, ATP-Latent, and ATP-Latent-SFT with different numbers of latent stages (i.e., number of latent tokens).

	#Stage	1	2	3	4	5	7	10	15	20
Coconut	Acc	25.2	33.1	35.6	34.7	33.8	30.7	26.5	21.0	12.7
	#Token	5.2	7.2	9.2	11.2	13.2	17.2	23.2	33.2	43.2
SIM-CoT	Acc	19.6	29.0	35.9	34.4	31.3	28.1	25.3	18.3	15.1
	#Token	5.2	7.2	9.2	11.2	13.2	17.2	23.2	33.2	43.3
ATP-Latent-SFT	Acc	12.4	25.1	34.0	39.1	39.7	39.8	40.0	40.3	40.2
	#Token	5.2	7.1	8.4	9.1	9.4	9.5	9.5	9.5	9.5
ATP-Latent	Acc	13.8	26.4	36.4	40.8	42.2	42.4	42.3	42.5	42.4
	#Token	5.1	7.1	8.5	9.3	9.7	9.8	9.8	9.8	9.9

D.3. Case Study: Exploring Different Ways

Section 6.3 shows that the RL process of ATP-Latent-SFT can actually develop new patterns, developing diverse reasoning paths, thus improving Pass@K (Yue et al., 2025a; Hu et al., 2026). In Figure 8, we decode the latent tokens to language steps and show the tree of the probability of going into language tokens representing different language steps. It directly exhibits that after RL training, ATP-Latent is able to generate a wider variety of valid reasoning paths (“branches”) for the same problem, rather than always generating latent paths representing one language label. This demonstrates that ATP-Latent’s RL stage encourages more generalizable and flexible reasoning by promoting exploration of multiple valid solutions.

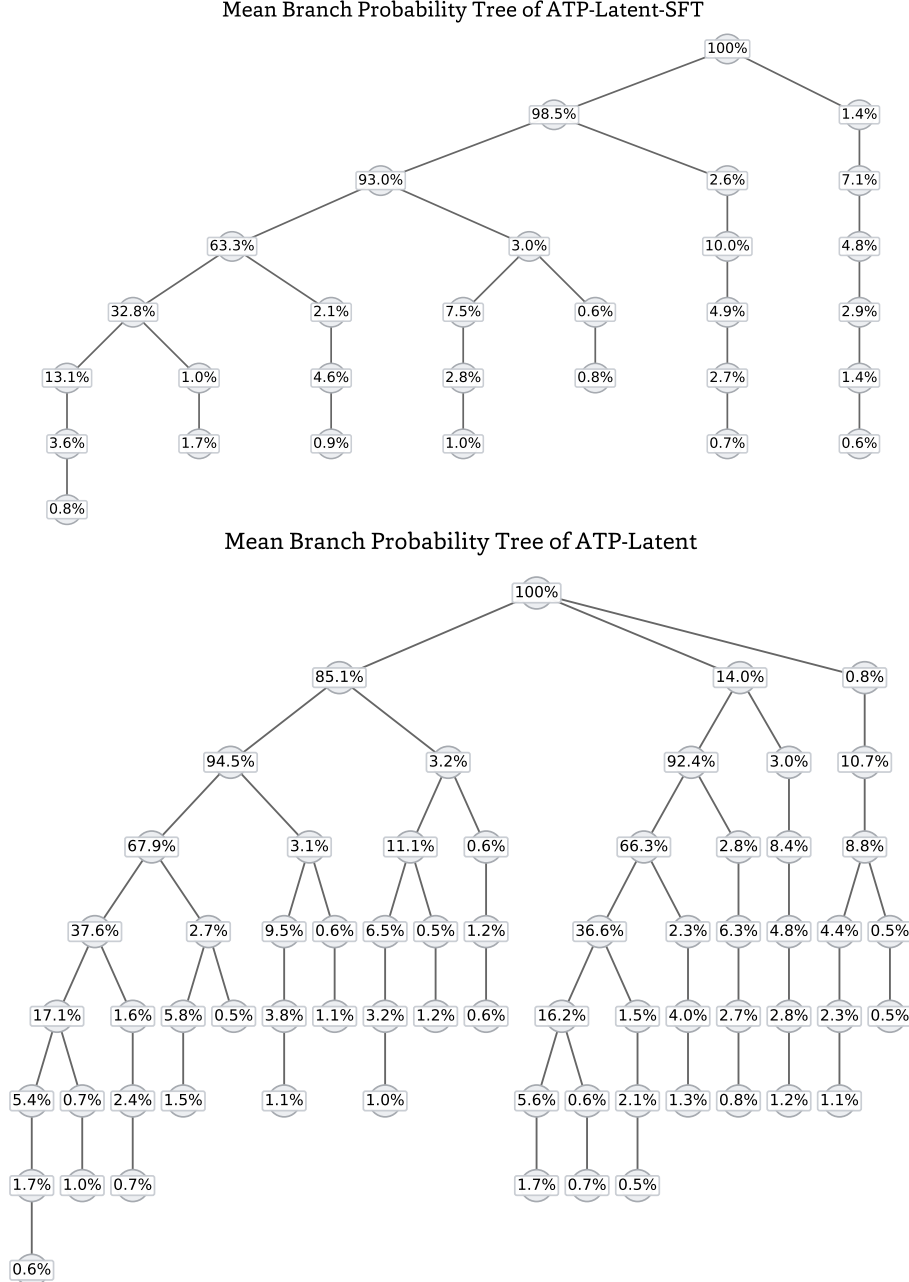


Figure 8. Pattern change after the RL training of ATP-Latent. The upper is for ATP-Latent-SFT, and the lower shows the pattern of ATP-Latent. We run each instance 64 times, decode each latent token to language steps by decoder, and count the probability of going into the top-1 branch or the branches with lower probabilities. Numbers in branches represent the probability of going into that branch. We do not show the branches with probability < 1% and the branches stopped by the stop head, so the summation of probabilities may not be 100%.

E. Baselines & Datasets & Licenses

E.1. Baselines

Based on the *LLaMA-3.2-1B-Instruct* LLM, we include SFT baselines for CoT-SFT, Answer-SFT. We also include latent reasoning baselines, CoLaR (Tan et al., 2025), iCoT (Deng et al., 2023), Coconut (Hao et al., 2024), and SIM-CoT (Wei et al., 2025)

SFT Baselines We implement SFT baselines based on the LLaMA-Factory (Zheng et al., 2024) framework. The prompts of CoT-SFT and Answer-SFT are given in Appendix B.

Latent Reasoning Baselines There may be complex training stages (e.g., SFT warm-up) in reproducing baselines (e.g., SIM-CoT); we fine-tune all latent reasoning from the base LLM using their given code. The reproduction results shown in Table 1 can generally match their report with ”*”.

E.2. Datasets

We follow (Wei et al., 2025) in adopting the GSM8K test set for in-domain, GSM-Hard (Gao et al., 2023), SVAMP (Patel et al., 2021), and Multi-Arith (Roy & Roth, 2015) for out-of-domain evaluation.

E.3. Licenses

For Base LLM, Dataset, and frameworks, we list their Licenses in Table 5.

Table 5. A summary of licenses.

Resources	Type	License	URL
LLaMA-3.2-1B-Instruct	Base LLM	Llama 3.2 License	https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct
LLaMA-Factory	SFT-framework	Apache-2.0 license	https://github.com/volcengine/verl
CoLaR	Baseline	Apache-2.0 license	https://github.com/xiaomi-research/cola
Coconut	Baseline	MIT License	https://github.com/facebookresearch/coconut
SIM-CoT	Baseline	Apache-2.0 license	https://github.com/InternLM/SIM-CoT
GSM8K, GSM-Hard, SVAMP	Dataset	MIT License	https://github.com/xiaomi-research/colar
Multi-Arith	Dataset	Available Online	https://github.com/xiaomi-research/colar
GSM8K-Aug	Dataset	Available Online	https://huggingface.co/datasets/whynlp/gsm8k-aug